

Системный дизайн как память агента

Поможет ли агенту база знаний
об архитектуре проекта?

Hacker Sprint #2

Эксперимент с кодинговым агентом и xmemory

Гипотеза: меньше разбираться заново

Каждая новая сессия снова изучает проект.

Мы решили сохранить найденные правила и передавать их агенту.

Например, задача

Исправить поиск рецептов:
слова с дефисами не находятся.

Что уже знает память

Нормализованные поля вычисляются
автоматически. Изменения требуют
миграции данных.

Ожидание: агент быстрее найдёт нужное место
и учтёт ограничения проекта.

Собрали скилл tech-facts

Человек выбирает,
что запоминать:
API, данные,
зависимости.

Агент извлекает факты из
кода. Сомнительные
отдаёт на ревью.

В интерфейсе видны
связи и ссылки на
исходники.

services-platform tech-facts Эксплорер

данные от 27 августа 2026 г. в 16:40 Система

services-platform
Эксплорер

ШАГ 1
Срезы
ждёт вашего выбора

ШАГ 2 · РЕВЬЮ3

Модули и зависимости
3 ждут вас

Граф событий
5 ждут вас

Владение данными
агент извлекает

ШАГ 3

Эксплорер

Поиск по фактам...

СТРАНИЦЫ

Обзор

События

Данные

Конфигурация

Грабли

СТАТУС ФАКТОВ

подтверждён авто

Граф событий

14 in-process событий через IEventBus из BuildingBlocks. Внешнего брокера нет — вся доставка синхронная, в транзакции публикатора.

Поток событий

копировать mermaid ↗

```
graph LR
    Pricing -- publishes --> priceUpdated[price.updated]
    priceUpdated -- notifies --> Orders
    priceUpdated -- notifies --> Catalog
    Orders -- publishes --> orderCreated[order.created]
    Orders -- publishes --> orderCancelled[order.cancelled]
    orderCreated -- notifies --> Billing
    orderCancelled -- notifies --> Billing
    Billing -- publishes --> paymentFailed[payment.failed]
    Billing -- publishes --> invoiceIssued[invoice.issued]
    paymentFailed -- notifies --> Notifications
    invoiceIssued -- notifies --> Notifications
```

Факты о событиях 9

Orders публикует order.created при успешном создании заказа

... уверенность: высокая fact:eg-0001

src/Modules/Orders/Application/CreateOrderHandler.cs:87

> детали

Billing подписан на order.created и создаёт черновик счёта

... уверенность: высокая fact:eg-0002

src/Modules/Billing/Application/OrderCreatedConsumer.cs:24

> детали

Notifications подписан на order.created и шлёт письмо-подтверждение

... уверенность: высокая fact:eg-0003

Память между задачами

В хmemory: типы фактов, сущности и связи между ними.

У факта есть источник в коде, статус и степень уверенности.

1. Перед задачей агент получает подходящие факты.
2. После правок обновляет память и сохраняет новые наблюдения.
3. Между задачами отдельная сессия убирает дубли и помечает устаревшие записи.

Для Mealie начали с **19 фактов**: API, ограничения, владение данными и настройки.

Как проверяли пользу

Mealie, 6 реальных задач: от исправления поиска до входа через OIDC. Claude Sonnet 5, effort medium.

Режим	Что получает агент
Без памяти	Код и постановку задачи
С памятью	Ещё и факты, которые обновляет после работы
С пересмотром памяти	То же + отдельную сессию ревизии фактов

Проверяли новые функции скрытыми тестами из реальных PR и отдельно проверяли, не сломалось ли старое поведение.

Сначала увидели экономию токенов

Первая проверка: один и тот же текст из 19 фактов для всех задач. 12 запусков.

-19%

выходных токенов с памятью

205 тыс. без памяти

166 тыс. с памятью

**Но 65% этой разницы
дала одна задача.**

На ней агент с памятью прошёл
1 из 9 проверок вместо 3 из 9
и сломал прежние тесты.

Меньший расход ещё не означает эффективность.
Нужно посмотреть, сколько работы агент выполнил.

В большом прогоне перевеса не было

Живое чтение и запись в хmemory. 6 задач × 3 режима × 2 повтора = 36 запусков.

Пройденные скрытые проверки	Повтор 1	Повтор 2
Без памяти	97 / 111	97 / 111
С памятью	91 / 111	97 / 111
С пересмотром памяти	97 / 111	95 / 111

Это наблюдения, а не доказательство вреда памяти.

По правилам отбора пригодны лишь 8 из 36 запусков:
агенты меняли существующие тесты или ломали старое поведение.

Убрали лишние факты. Качество то же

Следующая проверка: запретили менять старые тесты и сузили выдачу памяти. 9 запусков.

Доля подходящих фактов выросла с 28% до 100%
по медиане. Объём подсказки сократился примерно втрое.

Проверки новой функции	Без памяти	С памятью	С ревизией
Поиск рецептов	6 / 6	6 / 6	6 / 6
Вход через OIDC	3 / 5	3 / 5	3 / 5

На третьей задаче режим с ревизией сломал старое поведение.
Большой повторный эксперимент остановили.

Новые записи ещё не стали опытом

В большом прогоне все 24 сессии с памятью получали только исходные факты. Новые уроки не попадали в выдачу.

В следующей проверке один новый факт всё же дошёл:
урок о нормализации текста попал в задачу про счётчики рецептов.

Агент не отметил его как использованный.
Результат совпал с вариантом без памяти.

Мы подтвердили сохранение знаний между сессиями.
Пользу переноса урока на следующую задачу пока не показали.

Реляционная память добавила работы

Чтобы агент получил короткую подсказку, пришлось обслуживать целую модель проекта.

Что хотели получить	Что пришлось поддерживать
Правило из кода	Типы объектов, ключи и связи
Актуальную подсказку	Статусы, дубли и обновления после правок
Знание для новой задачи	Собственный отбор фактов и проверку их пользы

Наш вывод: для этой задачи сложность реляционной памяти пока не оправдалась результатом.

Что пришлось учитывать в хтemory

Запись текстом могла придумать структуру.

При обкатке получили 20 лишних связей и объект с пустым ключом. Перешли на явные структурированные изменения через API.

Выдачу под задачу пришлось делать самим.

Команда `context --text` возвращала общий контекст.

Добавили собственное ранжирование поверх сырых данных.

Историю операций собирали в раннере.

Логировали, что прочитали и записали. Денежную стоимость операций памяти CLI не сообщал.

Эвал тоже пришлось переделывать

Тест должен замечать отсутствие фичи.

Четыре из 23 задач-кандидатов отсеяли: их тесты проходили ещё до реализации. Проверка эталонного решения обязательна.

Тесты нужно защищать от агента.

В 22 из 36 запусков агент трогал существующие тесты.
В следующей версии запретили это на уровне окружения.

Экономию стоит искать в изучении кода.

Полезная память должна сокращать поиск нужного места.
Общее число токенов смешивает изучение, реализацию и ошибки.

Следующая гипотеза: помнить ошибки

Скилл, интерфейс и память между сессиями работают.
Преимущества в решении задач в этих прогонах не увидели.

**Дальше проверяли бы короткие уроки
из собственных ошибок агента.**

Для каждого урока: когда применять, что сделать,
какой тест подтвердил исправление.

Сравнили бы такую память в обычном файле и в хmemory
на задачах, где один и тот же урок нужен повторно.

Материалы и стоимость прогонов

Эксперимент	Запуски задач	Оценка usage
Полный текст фактов	12	\$31.90
Чтение, запись и ревизия	36	\$93.25
Точный отбор фактов	9	\$30.72

Около \$156 за эти три серии, включая сессии ревизии.
Без отладки и стоимости xmemory. Это оценка Claude Code по usage, а не сумма фактических списаний.

Отчёты, CSV и протокол эксперимента
Разбор экономии и метрик · Скилл и демонстрационный интерфейс